Автономная некоммерческая организация высшего образования
«Университет Иннополис»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)
по направлению подготовки
09.03.01 - «Информатика и вычислительная техника»

GRADUATION THESIS
(BACHELOR’S GRADUATION THESIS)
Field of Study
09.03.01 – “Computer Science”

Направленность (профиль) образовательной программы
«Анализ данных и искусственный интеллект»

Area of Specialization / Academic Program Title:
“Data Analysis and Artificial Intelligence”

Тема / Topic	**Надёжный код: сравнительный анализ методов оценки неопределённости для кодовых LLM / Code You Can Trust: Benchmarking Uncertainty Quantification Methods for Code LLMs**	
Работу выполнил / Thesis is executed by	**Софья Ткаченко / Sofia Tkachenko**	подпись / signature
Руководитель выпускной квалификационной работы / Supervisor of Graduation Thesis	**Владимир Иванов / Vladimir Ivanov**	подпись / signature
Консультанты / Consultants		подпись / signature

Иннополис, Innopolis, 2026

Contents

List of Tables

List of Figures

Abstract

Coding Large Language Models often produce confident but incorrect outputs. Uncertainty Quantification (UQ) methods flag likely failures, but research on UQ for code remains uncomparable across models, benchmarks, and metrics.

This thesis evaluates thirteen unsupervised UQ methods on HumanEval across three open-source code LLMs: DeepSeek-Coder-1.3B, DeepSeek-Coder-6.7B, and Qwen2.5-Coder-7B, comparing the top-ranked LM-Polygraph estimators against execution-based scores from functional and symbolic clustering. A shared greedy completion per problem ensures that PR-AUC and the Prediction Rejection Ratio (PRR) reflect uncertainty quality alone. By PR-AUC, the leading method is model-dependent: functional clustering ranks first and second on the larger DeepSeek, while CCP and MSP lead on Qwen. ROUGE-L and CCP appear in the top four by PR-AUC on the larger models. By PRR, the leading family differs across models, with sample-diversity methods on DeepSeek-6.7B and information-theoretic methods on Qwen, and both execution-based methods now ranking competitively. A recent optimization to symbolic clustering's implementation dramatically improved its performance, moving it from ranking last to 2nd–5th depending on the model and metric.

Chapter 1

Introduction

Large Language Models (LLMs) are now widely used to generate code. The Stack Overflow Developer Surveys [1] show adoption rising from 43.8% to 78.5% between 2023 and 2025, while the share of developers who distrust AI accuracy grew from 27.2% to 45.7% over the same period (Fig. 1.1). According to the 2025 survey [1], 87% of respondents are concerned about the accuracy of LLM outputs.

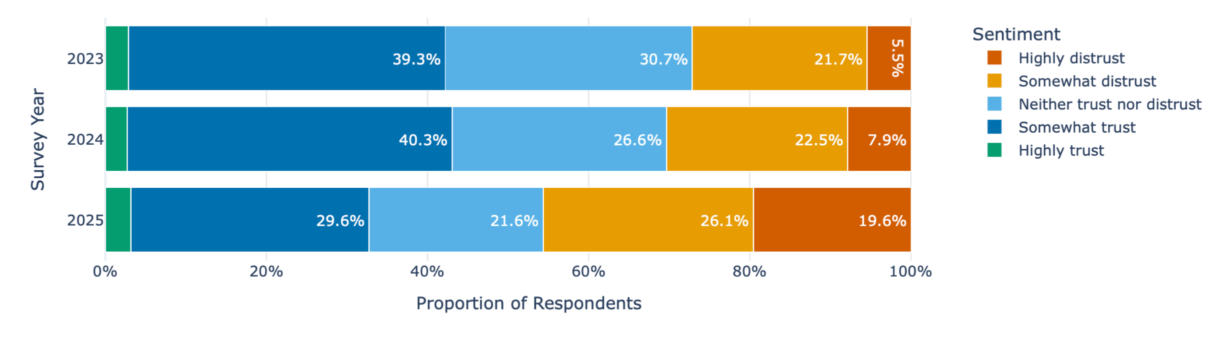

Fig. 1.1. Developer Trust in AI Accuracy (2023-2025) [1]

Developers are relying more on tools they trust less. The root cause for distrust is LLM hallucinations: models produce plausible but incorrect code. Uncertainty Quantification (UQ) methods can detect when a model is likely wrong, flagging suspicious outputs before they reach production. Nonetheless, most existing UQ methods were developed for natural language generation and do not transfer well to code. Recent studies [3], [4] show that standard text-similarity and token-probability methods have no statistically significant correlation with code correctness, while methods that run the generated programs perform far better. More about this finding is discussed in the Literature Review.

For natural language tasks, like question answering, UQ methods are already compared systematically through the LM-Polygraph benchmark [2]. Currently there is no such benchmark for code. Moreover, research in code UQ is mostly incomparable: different studies use different models, tasks, and metrics.

This thesis aims to fill that gap with two objectives:

- **First**, to evaluate the best-performing unsupervised lm-polygraph methods on code generation and determine how well they transfer from natural language.
- **Second**, to compare them against code-specific execution-based methods (functional clustering [4] and symbolic clustering [3]) on a shared benchmark.

The goal is an honest quantitative answer to which UQ strategies work best for code.

Chapter 2

Background

2.1 Hallucinations

Ji *et al.* [5] defined hallucinations as "generated content that is nonsensical or unfaithful to the provided source content". LLMs are prone to hallucinations because they are trained to predict the next most probable token. Training rewards plausible output for any prompt, even when the model lacks the information to answer correctly. If the model answers "I don't know" or produces an incomplete answer, it is penalized during training. As a result, the model learns to produce a complete answer regardless of confidence, leading to hallucinations.

There are also other sources of hallucinations. The model's knowledge is a compressed representation of its training data, and reconstruction from compressed patterns introduces errors and false associations. On the other hand, the training data itself can contain factual errors, outdated content, and contradictions, which the model can reproduce.

Code generation has its own hallucination problems. CodeMirage [6], HalluCode [7], CodeHalu [8], and Collu-Bench [9] show that LLMs frequently generate plausible-looking code with defects that are hard to detect: logical flaws, security vulnerabilities, unreachable code, or calls to non-existent libraries and functions [6].

2.2 Uncertainty and Confidence

There are two related concepts that are important for understanding and addressing hallucinations in LLMs: uncertainty and confidence.

- *Uncertainty* is a property of the input. Given a prompt x, the model's predicted distribution $P(Y|X = x)$ may be spread over many possible outputs or concentrated on a few. A vague prompt like “write a sort function” has many valid implementations, so the distribution is wide. A more precise prompt like “write a function that returns the sum of two integers” leaves less room for variation. Uncertainty depends on x only, not on any particular output [10].
- *Confidence* is a property of a specific prediction. Given input x and a generated output y, confidence measures how likely y is to be correct. A model can be uncertain about a prompt (many possible outputs) while still being confident in the one it chose [10].

However, in literature, the terms “uncertainty” and “confidence” are often used interchangeably. Often uncertainty is measured for each generated output, which is technically confidence. This thesis uses “uncertainty” to refer to the model's confidence in its output, following common usage in the field. As well as converts confidence scores to uncertainty scores by inverting them (uncertainty $= 1 -$ confidence) for consistency with lm-polygraph.

2.2.1 Types of Uncertainty

Uncertainty in LLMs comes from two sources [10]:

- *Epistemic uncertainty* (model uncertainty) appears from limitations of the model: gaps in training data or insufficient capacity. A model that never saw a particular type of problem during training will be epistemically uncertain about it. This type can in principle be reduced with more data or better training.
- *Aleatoric uncertainty* (data uncertainty) comes from inherent ambiguity of the task. Some problems have multiple correct solutions (e.g. "write a sorting function" can be solved with quicksort, mergesort, or insertion sort). This type of uncertainty cannot be reduced by improving the model because the ambiguity is in the problem.

Separating total uncertainty into these components is complex and usually unnecessary in practice [10]. This thesis measures total uncertainty without decomposing it.

2.3 Uncertainty Quantification

Uncertainty Quantification (UQ) is the set of methods that assign an uncertainty score to a model's output, so that highly uncertain generations can be rejected or handed off to a human for review. The idea comes from selective classification, where a classifier can abstain on inputs it cannot answer reliably rather than risk a wrong prediction [11], [12]. Abstention is important in domains like banking, healthcare, and law, where a deferred decision is far more preferable than a wrong one. The same logic applies to code: a flagged hallucination can be reviewed before it reaches production.

UQ methods first appeared in classification and regression [13], [14], based on information theory and Bayesian modeling [15]. They were later adapted to encoder language models such as BERT [16]. The scale and free-form output of modern text-generating LLMs required new approaches [17], leading to the current line of work on UQ for text generation [18]. UQ also underpins related tasks such as out-of-distribution detection [19], [20], defence against adversarial attacks [21], and active learning [13].

UQ methods differ in what access to the model they require [10]. **White-box** methods read internal signals such as logits or attention weights. **Black-box** methods use only the generated text, which makes them applicable to proprietary API-based models.

Chapter 3

Literature Review

3.1 UQ Method Categories

The LM-Polygraph benchmark [2] evaluates 29 UQ methods on natural language generation and organises them into four categories:

- **Information-Theoretic:** compute uncertainty from the model's token-level probability distribution. Examples include Maximum Sequence Probability, Perplexity, and Mean Token Entropy [22].
- **Sample Diversity:** generate multiple completions for the same prompt and measure their disagreement. Consistent outputs indicate higher confidence. Examples include Semantic Entropy [18], Degree Matrix, and Sum of Eigenvalues of the Graph Laplacian [10].
- **Density-based:** compare a generated output's embedding against a reference distribution of correct examples. Outputs far from the distribution are flagged as uncertain. Examples are Mahalanobis distance (MD) [23] and Robust density estimation (RDE) [24].
- **Reflexive:** ask the model to evaluate its own confidence directly (e.g. "Are you certain about your last response?"). The $p(\mathrm{True})$ method [25] is an example.

Each category comes with its strengths and weaknesses (according to [2]):

TABLE 1
Comparison of Uncertainty Quantification Method Categories

Category	**Access**	**Compute**	**Strengths**	**Weaknesses**
Information-Theoretic	White-Box	Low	Fast, theoretically grounded, provides token-level scores.	Requires internal model access. Can be naive to semantic meaning.
Sample Diversity	Black-Box	High	Model-agnostic. Captures semantic uncertainty.	Requires generating many outputs. Clustering can be complex and slow.
Density-Based	Black-Box	Medium	Good at detecting outputs that are structurally or stylistically unusual	Requires a high-quality reference dataset: performance depends heavily on the embedding space
Reflexive	Black-Box	Low	Simple to implement	Highly unreliable: models are often overconfident and poor at self-assessment

3.2 Code Hallucination Taxonomies

Several recent works classify what goes wrong when LLMs generate code. Each proposes a different taxonomy, but the defects they describe overlap. Understanding these categories is relevant for UQ because different types of hallucinations are caught by different detection strategies.

CodeMirage [6] collected 1,137 GPT-3.5-generated Python snippets from HumanEval and MBPP and annotated each for one of five defect types: logical errors (the code runs but solves the wrong problem), syntactic incorrectness (the code does not compile), dead or unreachable code (parts of the code serve no purpose), robustness issues (the code fails on edge cases or raises exceptions), and security vulnerabilities (the code has exploitable flaws or memory leaks). Each type appeared in roughly equal proportions in their dataset.

HalluCode [7] takes a different angle, classifying defects by what the code conflicts with rather than their technical nature. It covers Python and Java tasks and defines three top-level categories with twelve subcategories. Requirement conflicting (39.60%) means the code does something other than what was asked, with behaviour conflicting (35.40%) as its largest subcategory. Knowledge hallucinations (34.90%) means the code contradicts real-world knowledge such as library APIs (25.99%) or algorithmic conventions. Code inconsistency (25.50%) means the code has internal contradictions like undefined variables or useless statements. They find that model-related factors cause 80.86% of hallucinations and that larger models hallucinate less.

CodeHalu [8] focuses on the source of the error. Mapping hallucinations occur when the model incorrectly maps between the problem description and code variables or functions. Naming hallucinations use wrong identifiers. Resource hallucinations reference libraries or functions that do not exist. Logic hallucinations contain flawed algorithmic reasoning. These four categories are further divided into eight subcategories and verified by executing the generated code.

These taxonomies overlap but emphasise different aspects of the same underlying problem. Table 2 combines them into a unified overview:

TABLE 2

Code Hallucination Taxonomy

Hallucination Types	**Description**
• Logical Error [6] • Intent Conflicting [7] • Logic Hallucinations [8]	The code is syntactically valid but fails to meet the problem's requirements or contains flawed logic.
• Syntactic Incorrectness [6] • Dead or Unreachable Code [6] • Naming Hallucinations [8] • Context Deviation [7]	The code is malformed, uses wrong identifiers, contains non-functional or repetitive segments, or cannot be compiled.
• Robustness Issue [6]	The code compiles but fails at runtime on edge cases or lacks exception handling.
• Security Vulnerabilities [6] • Knowledge Conflicting [7] • Mapping Hallucinations [8] • Resource Hallucinations [8]	The code misuses variables, external APIs, or libraries, or calls non-existent resources, leading to errors or security flaws.

As Table 2 shows, code hallucinations are specific to the code domain. Methods from natural language generation (NLG) cannot be directly applied to code without adaptation.

3.3 Evaluating Uncertainty Quantification Methods

Measuring UQ performance is a challenge in itself. Some of the approaches used in literature are:

- Rank correlation (i.e. Spearman's ρ) between uncertainty scores and output quality metrics such as ROUGE or BLEU [22], [26]. This says little about practical performance, and n-gram metrics often miss semantic quality.
- Binary classification (correct vs. incorrect output) evaluated with AUROC or PR-AUC [2], [26], [27]. This requires an arbitrary correctness threshold, making cross-study comparison difficult.

This thesis uses both PR-AUC and the Prediction Rejection Ratio (PRR) [28], with PR-AUC as the primary metric. PRR is the standard in lm-polygraph [2] and earlier work [17]. Both are defined in Chapter 4.4.

3.4 Uncertainty Quantification for Code

UnCert-CoT [29] uses token entropy to switch from direct generation to Chain-of-Thought reasoning when uncertainty is high. AdaDec [30] uses token-level entropy to pause decoding and rerank candidate tokens.

Token-level signals often fail for code because syntactically different programs can do the same thing. Recent work clusters outputs by behaviour rather than text.

Ravuri and Amarasinghe [4] propose *Functional Clustering*. They argue that text embeddings miss small errors, like a swapped operator, that break code. Their method generates programs and test inputs, runs them in a sandbox, and groups programs with identical input-output behaviour. The size of the largest group is the confidence score. On LiveCodeBench, this reduced error rates from 65% to 2%, while token-probability scores could not reliably separate correct from incorrect outputs.

Sharma and David [3] propose *Symbolic Clustering*, which goes beyond unit tests. Their method uses symbolic execution to treat inputs as abstract symbols and generate traces of the code's logic, grouping programs whose traces match. Testing several standard UQ approaches on code, they found Pearson correlations with correctness near zero: text-similarity methods reached $r = -0.04$ to -0.21 (all $p > 0.08$), and token log-probabilities $r = 0.09$ to 0.18 ($p > 0.38$), none statistically significant. Only symbolic clustering achieved meaningful correlations ($r = -0.40$ to -0.56, $p < 0.001$), with a false positive rate below 0.02%.

Both [4] and [3] are sample-diversity methods like Semantic Entropy [18], and are therefore computationally expensive.

Other approaches use classical code metrics as proxies: compiler feedback [31], static code quality analysis [32], and pass rates of generated unit tests [33].

Chapter 4

Methodology

4.1 Models

The evaluation uses three open-source instruction-tuned models: **DeepSeek-Coder-1.3B-Instruct**, **DeepSeek-Coder-6.7B-Instruct** [34], and **Qwen2.5-Coder-7B-Instruct** [35]. The two larger models are 7-billion-parameter models trained on code corpora and fine-tuned for instruction following. The DeepSeek-1.3B variant enables comparison of model size effects within the same architecture. Across the larger models, the focus is on controlling for parameter count while testing whether results generalise across model families; they differ in training data and chat template format. Base model variants were tried first but did not consistently recognise the task as code completion.

4.2 Dataset

All experiments use **HumanEval** @humaneval, a benchmark of 164 Python programming problems. Each problem consists of a function signature with type annotations, a docstring with example input-output pairs, and a hidden unit test suite. The model receives only the stub (signature and docstring) and must generate the function body. Correctness is determined by running the completion against the unit tests.

The prompt wraps the stub in a fenced Python block with an instruction to complete it without modifying the given code, following DeepSeek's HumanEval protocol [34]:

````
Please continue to complete the function. You are not allowed to
modify the given code and do the completion only. Please return
all completed function in a codeblock. Here is the given code to
do completion:

    ```python
    {code}
    ```
````

4.3 Uncertainty Estimation Methods

This evaluation covers only unsupervised methods, which produce uncertainty scores without any labelled calibration data. Supervised methods, such as those that fit a reference distribution of correct examples, are excluded.

Standard UQ methods from natural language generation transfer poorly to code: text-similarity and token-probability approaches show no significant correlation with code correctness [3]. The evaluation therefore pairs the best-performing lm-polygraph [2] methods with execution-based methods that assess behaviour directly.

Thirteen uncertainty scores are evaluated in total: nine from the **lm-polygraph** framework and four from two execution-based approaches, each producing two scores from the same cluster structure. Table 3 summarises them.

TABLE 3
Summary of evaluated methods. All methods except MSP and Perplexity use $N = 10$ stochastic completions. "Compute" reflects cost relative to a single greedy pass. Functional and symbolic clustering each produce two scores (CC and SE).

Method	Category	Access	Compute	Aux. Model
MSP	Information-Theoretic	White-box	Low	No
Perplexity	Information-Theoretic	White-box	Low	No
LexSim ROUGE-L	Sample Diversity	Black-box	High	No
LexSim BLEU	Sample Diversity	Black-box	High	No
DegMat Jaccard	Sample Diversity	Black-box	High	No
CCP	Information-Theoretic	White-box	High	DeBERTa
SAR	Sample Diversity	White-box	High	DeBERTa
TokenSAR	Information-Theoretic	White-box	High	DeBERTa
DegMat NLI	Sample Diversity	Black-box	High	DeBERTa
Functional Clustering	Execution-based	Black-box	High	No
Symbolic Clustering	Execution-based	Black-box	High	CrossHair

4.3.1 Method Selection

The nine lm-polygraph methods were chosen by ranking all unsupervised estimators in the framework by mean PRR across two models (Stable LM 2 12B and Mistral 7B v0.2) in the original benchmark [2]. The top ten by mean PRR were MSP, CCP, SAR, HUQ-MD, ROUGE-L, DegMat NLI, Perplexity, DegMat Jaccard, BLEU, and TokenSAR. HUQ-MD was excluded because it is supervised: it requires a reference distribution of correct examples to compute Mahalanobis distances. The remaining nine span the information-based and sample-diversity categories. Reflexive methods did not reach the top ten.

The remaining methods are grouped below by whether they require an auxiliary NLI model. The first group (MSP, Perplexity, Lexical Similarity, DegMat-Jaccard) operates directly on tokens and token probabilities. The second group (CCP, SAR, TokenSAR, DegMat-NLI) additionally uses DeBERTa for semantic agreement. The execution-based methods form a third group.

4.3.2 Token-Level Methods

These methods use only token-level log-probabilities from a single greedy pass and $N = 10$ stochastic samples. No auxiliary model is required.

Maximum Sequence Probability (MSP) is the product of the greedy token probabilities:

$$\mathrm{MSP} = \exp\left(\sum_t \log p\left(x_t \mid x_{\{<t\}}, c\right)\right) \tag{1}$$

where c is the prompt and t indexes generated tokens. Higher MSP means higher per-token confidence and lower uncertainty.

Perplexity normalises MSP by sequence length so scores are comparable across outputs of different length:

$$\mathrm{PPL} = \exp\left(-\frac{1}{T}\sum_{t} \log p\left(x_t \mid x_{\{<t\}}, c\right)\right) \tag{2}$$

Higher perplexity indicates higher uncertainty.

Lexical Similarity methods measure consistency across samples. The mean overlap between the greedy completion and each of the N stochastic samples is computed using ROUGE-L @rouge (longest common subsequence recall) and separately using BLEU @bleu (n-gram precision). A model that generates diverse completions for the same problem is considered more uncertain than one whose completions are consistent.

Degree Matrix-Jaccard (DegMat-Jaccard) measures consistency across all pairs of samples rather than against the greedy output. A graph is constructed with one node per sample and edge weights given by the Jaccard similarity of the two samples' token sets. The uncertainty score is derived from the spectral properties of the graph's degree matrix [10]: similar completions give low spectral spread, diverse ones give high.

4.3.3 NLI-Augmented Methods

These methods additionally pass pairs of completions through DeBERTa @deberta, a bidirectional transformer trained on natural language inference (NLI). Given two texts, it outputs probabilities for entailment, neutrality, and contradiction. Entailment probability serves as a proxy for semantic agreement between completions: an approximation when applied to code, but richer than token overlap.

Claim-Conditioned Probability (CCP) @ccp re-weights the greedy token probabilities using NLI entailment scores from the sampled alternatives. If many samples are semantically consistent with the greedy completion, its token probabilities are upweighted, lowering the uncertainty estimate.

SAR (Semantic Answer Replacement) @sar measures how sensitive the greedy output is to token substitution. For each token, semantically equivalent alternatives from the sampled completions are identified using DeBERTa, and the change in sequence probability from the substitution is recorded. Tokens whose replacement barely shifts probability are less informative; load-bearing tokens shift it more. High average sensitivity indicates higher uncertainty.

TokenSAR @sar uses the same per-token sensitivities as SAR but weights each by its contribution to the overall sequence probability before aggregation.

Degree Matrix-NLI (DegMat-NLI) applies the same graph-based framework as DegMat-Jaccard but uses NLI entailment probability as the edge weight instead of Jaccard token overlap [10], capturing semantic rather than surface agreement.

4.3.4 Execution-Based Methods

Functional clustering and symbolic clustering use neither token probabilities nor text similarity. Instead, they generate $N = 10$ stochastic completions per problem and group them by whether they produce the same behaviour when executed. The assumption: a model producing behaviourally equivalent completions is more likely to be correct than one whose completions diverge.

Each method partitions the N completions into equivalence classes, from which two uncertainty scores are computed. **Cluster Count (CC)** is:

$$u_{\mathrm{CC}} = 1 - \frac{|C_{\max}|}{N} \tag{3}$$

where $|C_{\max}|$ is the size of the largest class. CC is zero when all completions are equivalent and approaches one when every completion is in its own class.

Semantic Entropy (SE) is the Shannon entropy of the cluster size distribution under a uniform prior:

$$u_{\mathrm{SE}} = -\sum_{c} \frac{|c|}{N} * \log \frac{|c|}{N} \tag{4}$$

SE is zero when all completions fall in one cluster and is maximised when they spread evenly across many. [3] shows that CC and SE produce comparable results when clustering is accurate: the aggregation formula matters less than the quality of the equivalence relation. Both are reported for direct comparison.

[3] also proposes Mutual Information (MI), which queries the model twice per problem with the second prompt formed by appending the first response to the original. MI is excluded here because its two-call inference setup produces structurally different completions, which would make pass@1 incomparable across methods.

The two execution-based methods use the same CC and SE formulas and differ only in how equivalence is determined.

Functional Clustering

Proposed by [4], this method determines equivalence by running each completion on concrete test inputs and grouping completions that produce identical outputs on every input. Test inputs are generated by prompting the LLM itself given only the function signature, so the method is self-contained and works on benchmarks without ground-truth tests.

The limitation is that equivalence is tested only on a finite set of inputs. Two functions that agree on every provided input but differ elsewhere will be incorrectly merged, underestimating uncertainty.

Symbolic Clustering

Proposed by [3], this method determines equivalence using symbolic execution rather than concrete inputs. The goal is to find a counterexample (a concrete input on which two functions differ) by reasoning over all possible inputs at once.

This is done using CrossHair @crosshair, a Python symbolic execution engine backed by an SMT solver. For each of the $\frac{N(N-1)}{2}$ pairs of completions, CrossHair explores both functions' execution paths with symbolic inputs and asks whether any input assignment causes them to diverge. If one is found, the pair goes into separate clusters. If no counterexample is found before the timeout, the functions are declared equivalent and merged via union-find, which ensures transitivity: if $f_i \equiv f_j$ and $f_j \equiv f_k$, all three are placed in the same cluster.

The limitation is that symbolic execution is bounded: CrossHair explores paths only up to a configurable depth. Functions equivalent at shallow depth but diverging at deeper paths may be incorrectly merged.

4.4 Evaluation Metrics

Each method produces a scalar uncertainty score $u_i \in \mathbb{R}$ and a binary correctness label $c_i \in \{0, 1\}$ per problem, where $c_i = 1$ if the greedy completion passes all unit tests. Three metrics are reported.

Pass@1 is the fraction of problems solved:

$$\text{pass@1} = \frac{1}{M} \sum_{i=1}^{M} c_i, \quad M = 164 \tag{5}$$

All methods share the same greedy completion, so pass@1 is identical across all thirteen scores for a given model. It characterises model capability rather than uncertainty estimation quality.

Prediction Rejection Ratio (PRR) [28] measures how well uncertainty scores identify incorrect predictions. Problems are ranked by decreasing uncertainty and progressively withheld; at each threshold, accuracy is computed on the remaining ones. PRR is the area between this curve and the random-rejection baseline (a horizontal line at pass@1), normalised by the area between the oracle curve (which rejects all incorrect predictions first) and the same baseline:

$$\mathrm{PRR} = \frac{\mathrm{AUC}_{\mathrm{uq}} - \mathrm{AUC}_{\mathrm{random}}}{\mathrm{AUC}_{\mathrm{oracle}} - \mathrm{AUC}_{\mathrm{random}}} \tag{6}$$

PRR = 1 means perfect failure identification; PRR = 0 means random ranking. Fig. 4.1 illustrates the relationship between these curves.

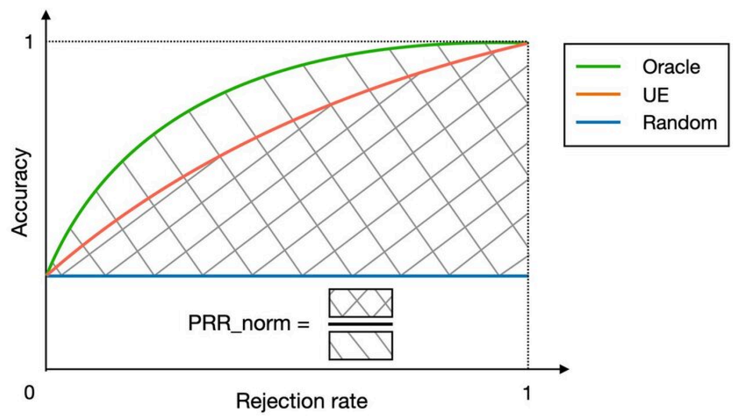

Fig. 4.1. Prediction-rejection curve schematic: oracle, uncertainty estimator, and random baseline [2].

PR-AUC is the area under the precision-recall curve, treating failures ($c_i = 0$) as the positive class and uncertainty as the detection score. Precision is the fraction of flagged predictions that are incorrect; recall is the fraction of all incorrect predictions that are flagged. PR-AUC aggregates this trade-off across all thresholds.

PR-AUC is the primary metric of this thesis. The intended use of code UQ is a threshold decision: a generated function is flagged for review or accepted. PR-AUC measures the precision-recall trade-off at every threshold, while PRR measures the full ranking of problems by uncertainty, independent of any specific threshold. With pass@1 of 0.689 and 0.726, failures are the minority class, where PR-AUC is preferred over ROC-based measures.

PRR is reported alongside because it is the standard metric in lm-polygraph [2] and most natural-language UQ work, and because it is scale-invariant against pass@1 differences across models. The two metrics can disagree: methods with coarse uncertainty scores (those taking only a few distinct values, as in cluster-based methods where the score is determined by cluster count or size) can score high on PR-AUC while ranking lower on PRR. Chapter 7.3 examines this divergence.

Chapter 5

Experimental Design

5.1 Setup

5.1.1 Inference

All inference runs on a single A100 GPU using vLLM @vllm with eager execution, `bfloat16` precision, and CUDA graph compilation disabled. vLLM is initialised at 65% GPU memory utilisation, leaving room for DeBERTa in the same process.

Fair comparison across the nine lm-polygraph estimators required care. An earlier design ran Group 1 and Group 2 in separate processes at 85% and 65% memory utilisation, which caused 5% of greedy completions to differ between groups. The cause is not random floating-point error: `gpu_memory_utilization` controls the size of the KV cache, which determines how attention is chunked, which in turn changes floating-point operation ordering and produces different logits. The resulting pass@1 gap confounds PR-AUC and PRR, since both metrics are sensitive to the fraction of correct predictions.

To avoid this, both groups run in one model instance at 65% utilisation. Greedy decoding happens once per problem; all estimators read the resulting token sequence from a shared dependency dictionary. The four execution-based scores adopt the same greedy completion via a `--greedy-file` argument, so all thirteen uncertainty scores share identical pass@1 by construction.

5.1.2 Prompt Construction

Each HumanEval stub is wrapped in a task instruction adapted from the DeepSeek-Coder evaluation protocol @deepseek-coder-eval and passed through each model's native chat template via `tokenizer.apply_chat_template` (DeepSeek uses `### Instruction / ### Response` delimiters, Qwen uses ChatML). The model is asked to return the completed function in a fenced code block; the body is extracted by stripping the fence markers and the echoed stub prefix.

5.1.3 Sampling

Each problem receives one greedy completion and $N = 10$ stochastic completions at temperature 1.0. These samples are reused by every method: lm-polygraph methods read them from a `_samples.jsonl` file written during inference, and the clustering methods use the same file (prepending the HumanEval stub to reconstruct complete function definitions). No additional model calls are needed for either clustering approach.

5.2 Functional Clustering

Test inputs are generated per problem by prompting the same model on the function signature and docstring alone, following [4]. The response is parsed as a JSON array of parameter-to-value dictionaries.

The reference implementation for HumanEval deviated from this design: it extracted inputs from the dataset's built-in `assert candidate(...)` statements (the same inputs used by `evaluate_functional_correctness`), which gave the method access to the evaluation criterion. This evaluation restores the paper's design by using only LLM-generated inputs.

Each body-only sample is prepended with the original stub to form a complete function, then run on each input with a one-second timeout enforced via `func_timeout`. Two completions are merged only if they produce the same output on every input; any completion that raises an exception or times out on any input is placed in an isolated cluster. The completion written to the output file is the greedy completion from the shared lm-polygraph run, converted back to body-only format.

5.3 Symbolic Clustering

For each problem, all $\frac{N(N-1)}{2} = 45$ pairs of completions are checked using CrossHair's `diffbehavior` command @crosshair. Each pair is written to a temporary Python module with the two functions renamed to `fn_a` and `fn_b`:

```
python -m crosshair diffbehavior cmp_module.fn_a cmp_module.fn_b
\
    --per_condition_timeout=10 --per_path_timeout=10
```

If CrossHair finds a concrete counterexample, the pair goes to separate clusters; no output means no counterexample was found within the timeout, and the pair is merged via union-find. Two edge cases are handled explicitly: syntactically invalid functions are placed in isolated clusters rather than merged by default, and pairs exceeding a hard wall-clock limit (three times the per-condition timeout) are treated as equivalent, consistent with the bounded-search guarantee. With 45 pairs across 164 problems, the total is 7,380 CrossHair calls per model.

5.4 Evaluation Protocol

Functional correctness is assessed with `evaluate_functional_correctness` from the HumanEval release @humaneval. Because this harness executes arbitrary model-generated code, it runs inside a Docker container with `--network none` to block outbound network access from generated programs. Each of the thirteen output files is evaluated independently, producing a `_results.jsonl` with a `passed` field per problem. PR-AUC and PRR are computed from the paired (uncertainty, correctness) vectors.

Chapter 6

Results

6.1 Pass@1

TABLE 4

Pass@k on HumanEval (164 problems). All thirteen uncertainty methods share pass@1 per model by construction. Unbiased pass@1 is the standard estimator.

Model	pass@1	pass@10
DeepSeek-Coder-6.7B-Instruct	0.689 (0.718 unbiased)	0.927
Qwen2.5-Coder-7B-Instruct	0.726 (0.753 unbiased)	0.957

6.2 PR-AUC and PRR

TABLE 5

PR-AUC and PRR for all thirteen uncertainty scores on DeepSeek-Coder-6.7B-Instruct, sorted by PR-AUC. All methods share pass@1 = 68.9%.

Method	PR-AUC	PRR
Functional Clustering SE	0.6068	0.8038
Functional Clustering CC	0.5935	0.8034

Method	**PR-AUC**	**PRR**
Lexical Similarity ROUGE-L	0.5854	0.8299
Claim-Conditioned Probability	0.5832	0.7751
Maximum Sequence Probability	0.5553	0.7608
DegMat Jaccard	0.5483	0.8103
SAR	0.5431	0.8325
Lexical Similarity BLEU	0.5280	0.8081
Perplexity	0.4389	0.7386
TokenSAR	0.4378	0.7373
DegMat NLI	0.3449	0.7285
Symbolic Clustering SE	0.3120	0.5968
Symbolic Clustering CC	0.2954	0.5520

On DeepSeek, functional clustering SE and CC lead by PR-AUC (0.607 and 0.594), followed by ROUGE-L (0.585) and CCP (0.583). The PRR ordering is different: the top four are sample-diversity methods (SAR 0.833, ROUGE-L 0.830, DegMat-Jaccard 0.810, BLEU 0.808), with functional clustering CC and SE fifth and sixth at PRR 0.80. Information-theoretic methods (CCP, MSP, Perplexity, TokenSAR) are mid-pack on both metrics. DegMat-NLI ranks eleventh on both. Symbolic clustering is last on both metrics, with PRR 0.55–0.60 and PR-AUC below 0.32.

TABLE 6

PR-AUC and PRR for all thirteen uncertainty scores on Qwen2.5-Coder-7B-Instruct, sorted by PR-AUC. All methods share pass@1 = 72.6%.

Method	**PR-AUC**	**PRR**
Claim-Conditioned Probability	0.6433	0.8581
Maximum Sequence Probability	0.5570	0.8586
Lexical Similarity ROUGE-L	0.4800	0.8180

Method	PR-AUC	PRR
Lexical Similarity BLEU	0.4566	0.8182
Perplexity	0.4521	0.8402
TokenSAR	0.4483	0.8399
DegMat Jaccard	0.4421	0.8138
SAR	0.4368	0.7996
Functional Clustering SE	0.4004	0.8359
DegMat NLI	0.3968	0.7971
Functional Clustering CC	0.3905	0.8352
Symbolic Clustering SE	0.3381	0.7109
Symbolic Clustering CC	0.3109	0.7352

On Qwen, CCP (0.643) and MSP (0.557) lead by PR-AUC, followed by ROUGE-L (0.480) and BLEU (0.457). The PRR top four are also information-theoretic methods (MSP 0.859, CCP 0.858, Perplexity 0.840, TokenSAR 0.840). Functional clustering ranks fifth and sixth by PRR (0.836) but only ninth to eleventh by PR-AUC (0.39–0.40), the opposite of its DeepSeek result. DegMat-NLI is near the bottom on both metrics. Symbolic clustering is last on both, with PRR 0.71–0.74 and PR-AUC 0.31–0.34.

6.2.1 Prediction-Rejection Curves

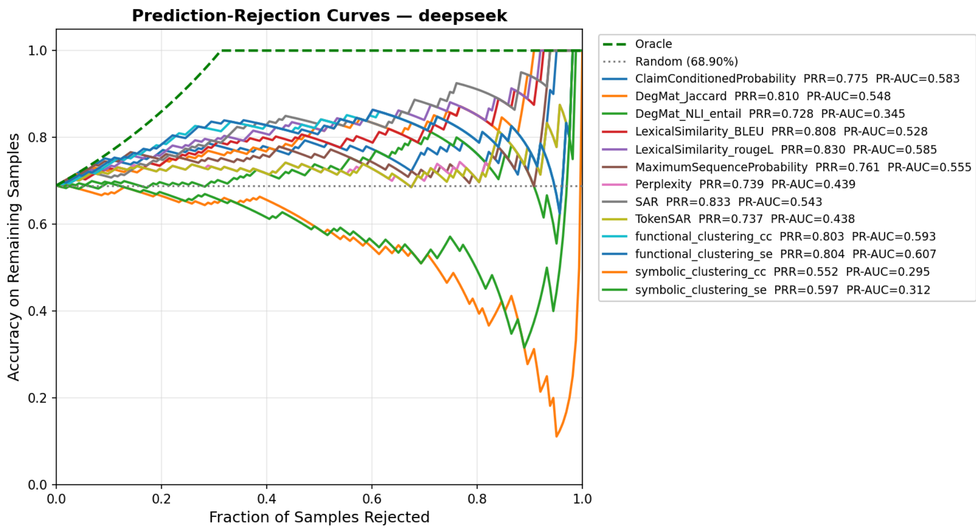

Fig. 6.1. Prediction-rejection curves for all methods on DeepSeek-Coder-6.7B-Instruct.

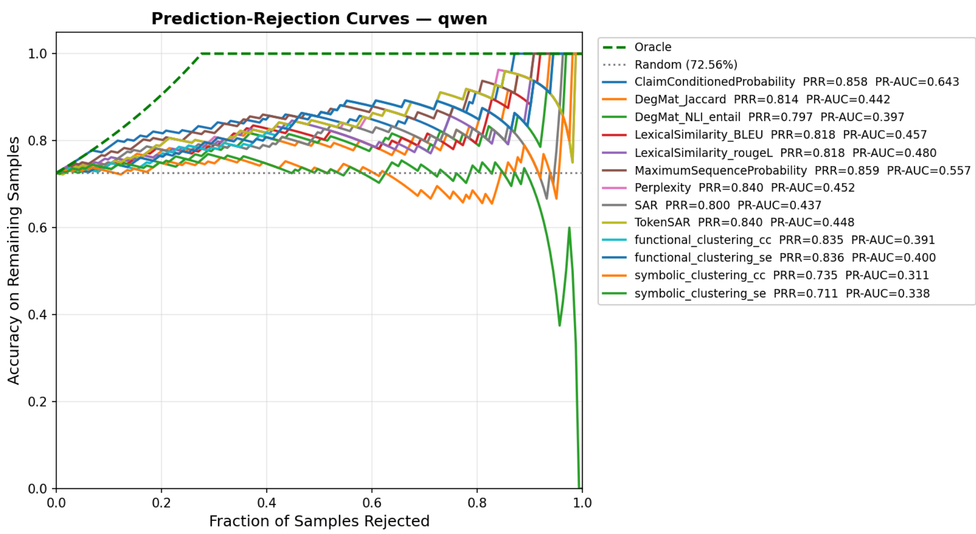

Fig. 6.2. Prediction-rejection curves for all methods on Qwen2.5-Coder-7B-Instruct.

6.2.2 Precision-Recall Curves

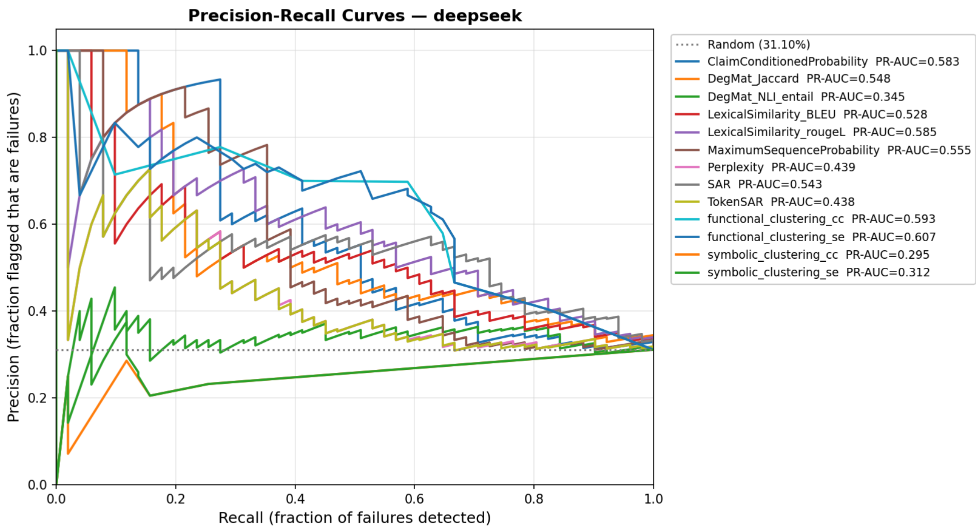

Fig. 6.3. Precision-recall curves (failure detection) for all methods on DeepSeek-Coder-6.7B-Instruct.

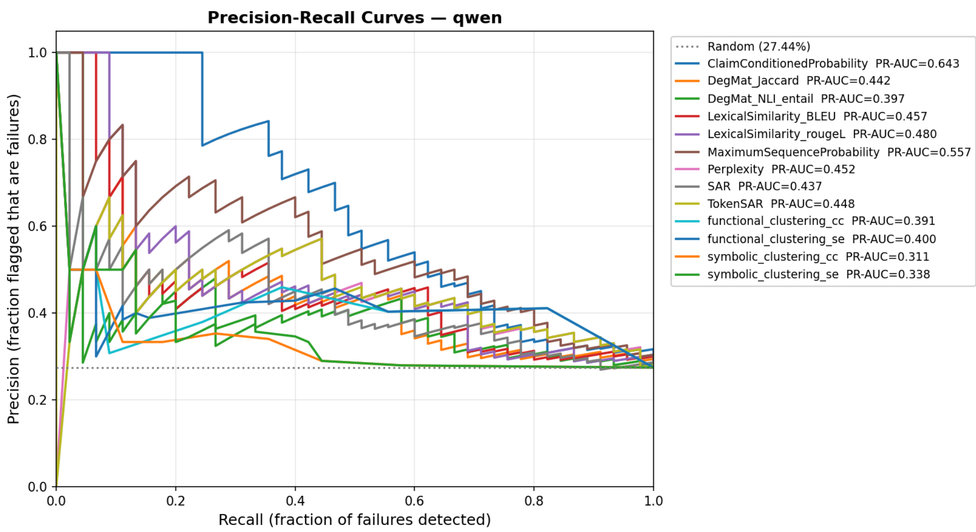

Fig. 6.4. Precision-recall curves (failure detection) for all methods on Qwen2.5-Coder-7B-Instruct.

6.3 Symbolic Clustering: Cluster Distribution

On DeepSeek, 108 out of 164 problems (65.9%) had all 10 completions merged into a single cluster (CC = 0, SE = 0); only 56 (34.1%) had more than one cluster. Mean CC across all problems was 0.086 and mean SE was 0.219. Functional clustering produces a wider spread, because concrete test inputs can distinguish functions that CrossHair's bounded search cannot. Chapter 7.3 discusses the consequences for PR-AUC and PRR.

6.4 Summary

No single method leads on both models. By PR-AUC, functional clustering ranks first and second on DeepSeek, while CCP and MSP take those positions on Qwen. ROUGE-L and CCP are the only methods in the top four by PR-AUC on both. By PRR, the four sample-diversity methods take the top spots on DeepSeek and the four information-theoretic methods take them on Qwen, with functional clustering fifth and sixth on both. Symbolic clustering ranks last by both metrics on both models. PR-AUC and PRR rank methods differently; Chapter 7.3 explains why.

Chapter 7

Discussion

7.1 Symbolic Clustering Timeout Sensitivity

Symbolic clustering ranked last despite strong results reported in [3]. The cause is CrossHair's bounded search: with a 10-second per-condition timeout, CrossHair could not find counterexamples for most function pairs on HumanEval, and absent a counterexample the pair was merged. As a result, 65.9% of problems had all completions in a single cluster (CC = 0, SE = 0), leaving the method unable to separate correct from incorrect predictions.

[3] reports $r = -0.40$ to -0.56 ($p < 0.001$), but on different models, different problems, and possibly longer timeouts. HumanEval functions often involve complex types (nested lists, dictionaries, strings with special characters) that make symbolic path exploration expensive. A longer timeout would likely improve cluster quality, but at substantial cost: the current 7,380 calls already take several hours.

This does not invalidate the method. It shows that performance depends on the timeout budget and function complexity: when CrossHair has time to explore paths, it can detect differences that concrete tests miss; when it times out, it defaults to merging everything.

7.2 Functional Clustering with LLM-Generated Test Inputs

Running with LLM-generated test inputs (as [4] describes, not as the reference implementation did) dropped functional clustering's PRR from 0.87 to 0.80 on DeepSeek and 0.84 on Qwen. The magnitude of the drop shows how much of the earlier score came from test-data overlap rather than the method's own signal.

Fairly compared, functional clustering remains competitive but no longer dominates. By PRR it ranks fifth on both models, behind sample-diversity methods on DeepSeek (SAR, ROUGE-L) and information-based methods on Qwen (MSP, CCP). By PR-AUC the picture is model-dependent: it ranks first and second on DeepSeek, but ninth to eleventh on Qwen. Execution-based clustering provides signal comparable to the better black-box estimators when test inputs are independent of the evaluation criterion.

7.3 PR-AUC and PRR Disagree

The two metrics measure different aspects of an uncertainty estimator. PRR depends on the full ranking of problems by uncertainty: it is the area between the rejection curve and the random baseline, normalised against the oracle. PR-AUC aggregates precision across recall thresholds, so it depends on whether incorrect predictions receive higher uncertainty than correct ones, not on the order within each group.

Score distribution drives the disagreement. Cluster-based estimators such as functional clustering CC and SE produce few distinct values, bounded by $N = 10$, and many problems share the same score. Ties depress PRR, since tied problems cannot be ordered against each other, but they do not affect PR-AUC if the tied scores fall on the correct side of the threshold. Continuous estimators such as ROUGE-L exhibit the opposite trade-off: smooth rankings yield strong PRR, while the precision-recall trade-off can be flatter. On DeepSeek, SAR ranks first by PRR (0.833) but seventh by PR-AUC (0.543), while functional clustering SE ranks first by PR-AUC (0.607) and fifth by PRR (0.804).

The two metrics suit different settings. PRR fits use cases where uncertainty is used to rank candidates, such as a review queue. PR-AUC fits use cases where failures are flagged at a fixed threshold, such as deployment.

Chapter 8

Conclusion

Code LLMs produce hallucinations that are hard to detect. Uncertainty quantification can flag likely failures before they reach production, but research on UQ for code is fragmented: studies use different benchmarks, models, and metrics, so cross-study comparisons are hard. This thesis evaluated thirteen unsupervised uncertainty estimators on HumanEval, on two open-source code LLMs of similar size: DeepSeek-Coder-6.7B-Instruct and Qwen2.5-Coder-7B-Instruct.

All thirteen estimators share the same greedy completion per problem, so pass@1 is identical across methods (68.9% on DeepSeek, 72.6% on Qwen) and PR-AUC and PRR reflect only uncertainty quality, not differences in model capability.

8.1 Key findings

By PR-AUC, no single method leads on both models. Functional clustering with LLM-generated test inputs ranks first and second on DeepSeek but ninth to eleventh on Qwen. CCP and MSP lead on Qwen and remain in DeepSeek's top four. ROUGE-L and CCP are the only methods in the top four by PR-AUC on both models.

By PRR, the leading family also differs by model: sample-diversity methods take the top four on DeepSeek (SAR, ROUGE-L, DegMat-Jaccard, BLEU), information-theoretic methods take them on Qwen (MSP, CCP, Perplexity, TokenSAR), with functional clustering fifth on both. The two metrics disagree systematically; Chapter 7.3 attributes this to score-distribution differences across method families.

Symbolic clustering ranks last on both metrics and both models, because CrossHair times out and merges most completions into a single cluster, leaving the score unable to separate correct from incorrect outputs.

The compute cost of execution-based methods depends on the task: NP-hard problems or factorial-time solutions can produce extremely long runtimes. Timeouts bound this in practice, since both clustering methods fall back to merging on timeout. The trade-off is that each timeout collapses potentially distinct completions into one cluster, degrading UQ quality (symbolic clustering's last-place ranking is a direct consequence). Compute-heavy lm-polygraph methods have predictable cost but no equivalent escape, since partial computation does not return a usable uncertainty estimate.

8.2 Contributions

- A shared-inference pipeline that gives identical pass@1 across all thirteen UQ methods on two code LLMs, so PR-AUC and PRR comparisons are not affected by differences in model capability.
- A correction to the public functional-clustering reference implementation, which used HumanEval's own assertions as test inputs instead of generating them via the LLM as [4] describes. With independent inputs, PRR drops from 0.87 to 0.80 on DeepSeek, so part of the earlier advantage came from reusing the evaluation tests.

- The first direct comparison via PR-AUC and PRR of the top lm-polygraph estimators against execution-based clustering on a shared code benchmark, which shows that the ranking depends on the model.

8.3 Limitations

The study uses one benchmark (HumanEval) and two ~7B models from circa 2024 [34], [35]. Both receive the same prompt, adapted from the DeepSeek-Coder HumanEval evaluation protocol [34], which may favour DeepSeek. The evaluation covers only unsupervised methods, so supervised approaches such as HUQ-MD are not represented. All stochastic sampling uses $N = 10$ completions at temperature 1.0. Other temperatures, top-p settings, and unbiased pass@k were not explored. No post-hoc calibration was applied to the uncertainty scores. Calibrating the scores (for example with temperature scaling on a held-out set [36]) could improve absolute PR-AUC and PRR values, although the relative ranking of methods is expected to be less affected. Symbolic clustering runs at one CrossHair budget (10 seconds per condition). The NLI-augmented methods (CCP, SAR, TokenSAR, DegMat-NLI) rely on DeBERTa, trained on natural-language entailment, which only approximates semantic agreement on code.

8.4 Future work

Extending the benchmark to other datasets (MBPP, LiveCodeBench, HumanEval-X) and to models of different sizes would test how well these rankings generalise. A larger CrossHair budget could improve symbolic clustering performance. The split between sample-diversity and information-theoretic methods across the two models suggests that hybrid scores combining token-level and execution-level signals may do better than either family alone.

Bibliography cited

[1] Stack Overflow, “Stack Overflow Developer Survey.” [Online]. Available: https://survey.stackoverflow.co/

[2] R. Vashurin *et al.*, “Benchmarking Uncertainty Quantification Methods for Large Language Models with LM-Polygraph,” *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 220–248, 2025, doi: 10.1162/tacl_a_00737.

[3] A. Sharma and C. David, “Assessing Correctness in LLM-Based Code Generation via Uncertainty Estimation,” *ArXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2502.11620

[4] C. Ravuri and S. Amarasinghe, “Eliminating Hallucination-Induced Errors in LLM Code Generation with Functional Clustering,” 2025, [Online]. Available: https://arxiv.org/abs/2506.11021

[5] Z. Ji *et al.*, “Survey of Hallucination in Natural Language Generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, Mar. 2023, doi: 10.1145/3571730.

[6] V. Agarwal *et al.*, “CodeMirage: Hallucinations in Code Generated by Large Language Models,” *ArXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2408.08333

[7] F. Liu *et al.*, “Exploring and Evaluating Hallucinations in LLM-Powered Code Generation,” *ArXiv*, 2024, [Online]. Available: https://arxiv.org/abs/2404.00971

[8] Y. Tian *et al.*, "CodeHalu: investigating code hallucinations in LLMs via execution-based verification," in *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*, in AAAI'25. AAAI Press, 2025. doi: 10.1609/aaai.v39i24.34717.

[9] N. Jiang *et al.*, "Collu-Bench: A Benchmark for Predicting Language Model Hallucinations in Code," *ArXiv*, 2024, [Online]. Available: https://arxiv.org/abs/2410.09997

[10] Z. Lin *et al.*, "Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models," *Transactions on Machine Learning Research*, 2024, [Online]. Available: https://openreview.net/forum?id=DWkJCSxKU5

[11] C. Chow, "On optimum recognition error and reject tradeoff," *IEEE Transactions on Information Theory*, vol. 16, no. 1, pp. 41–46, 1970, doi: 10.1109/TIT.1970.1054406.

[12] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, in NIPS'17. Curran Associates Inc., 2017, pp. 4885–4894. doi: 10.5555/3295222.3295241.

[13] Y. Gal *et al.*, "Deep Bayesian Active Learning with Image Data," in *Proceedings of the 34th International Conference on Machine Learning*, D. Precup and Y. W. Teh, Eds., in Proceedings of Machine Learning Research, vol. 70. PMLR, 2017, pp. 1183–1192. [Online]. Available: https://proceedings.mlr.press/v70/gal17a.html

[14] B. Lakshminarayanan *et al.*, "Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles," *ArXiv*, 2017, [Online]. Available: https://arxiv.org/abs/1612.01474

[15] C. Blundell *et al.*, "Weight Uncertainty in Neural Network," in *Proceedings of the 32nd International Conference on Machine Learning*, F. Bach and D. Blei, Eds., in Proceedings of Machine Learning Research, vol. 37. Lille, France: PMLR, 2015, pp. 1613–1622. [Online]. Available: https://proceedings.mlr.press/v37/blundell15.html

[16] A. Shelmanov *et al.*, "Active Learning for Sequence Tagging with Deep Pre-trained Models and Bayesian Uncertainty Estimates," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, P. Merlo, J. Tiedemann, and R. Tsarfaty, Eds., Online: Association for Computational Linguistics, Apr. 2021, pp. 1698–1712. doi: 10.18653/v1/2021.eacl-main.145.

[17] A. Malinin and M. Gales, "Uncertainty Estimation in Autoregressive Structured Prediction," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=jN5y-zb5Q7m

[18] L. Kuhn *et al.*, "Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation," *ArXiv*, 2023, [Online]. Available: https://arxiv.org/abs/2302.09664

[19] A. Podolskiy *et al.*, "Revisiting mahalanobis distance for transformer-based out-of-domain detection," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 15, pp. 13675–13682, 2021, doi: 10.1609/aaai.v35i15.17612.

[20] J. Ren *et al.*, "Out-of-Distribution Detection and Selective Generation for Conditional Language Models," in *The Eleventh International Conference on*

Learning Representations, 2023. [Online]. Available: https://openreview.net/forum?id=kJUS5nD0vPB

[21] L. Smith and Y. Gal, "Understanding Measures of Uncertainty for Adversarial Example Detection," *ArXiv*, 2018, [Online]. Available: https://arxiv.org/abs/1803.08533

[22] M. Fomicheva *et al.*, "Unsupervised Quality Estimation for Neural Machine Translation," *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 539–555, 2020, doi: 10.1162/tacl_a_00330.

[23] K. Lee *et al.*, "A Simple Unified Framework for Detecting Out-of-Distribution Samples and Adversarial Attacks," *ArXiv*, 2018, [Online]. Available: https://arxiv.org/abs/1807.03888

[24] K. Yoo *et al.*, "Detection of Adversarial Examples in Text Classification: Benchmark and Baseline via Robust Density Estimation," in *Findings of the Association for Computational Linguistics: ACL 2022*, S. Muresan, P. Nakov, and A. Villavicencio, Eds., Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 3656–3672. doi: 10.18653/v1/2022.findings-acl.289.

[25] S. Kadavath *et al.*, "Language Models (Mostly) Know What They Know," *ArXiv*, 2022, [Online]. Available: https://arxiv.org/abs/2207.05221

[26] T. Abbasli *et al.*, "Comparing Uncertainty Measurement and Mitigation Methods for Large Language Models: A Systematic Review.," *CoRR*, Apr. 2025, [Online]. Available: http://dblp.uni-trier.de/db/journals/corr/corr2504.html#abs-2504-18346

[27] C. Ling *et al.*, "Uncertainty Quantification for In-Context Learning of Large Language Models," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human*

Language Technologies (Volume 1: Long Papers), K. Duh, H. Gomez, and S. Bethard, Eds., Mexico City, Mexico: Association for Computational Linguistics, June 2024, pp. 3357–3370. doi: 10.18653/v1/2024.naacl-long.184.

[28] A. Malinin *et al.*, "Incorporating Uncertainty into Deep Learning for Spoken Language Assessment," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, R. Barzilay and M.-Y. Kan, Eds., Vancouver, Canada: Association for Computational Linguistics, July 2017, pp. 45–50. doi: 10.18653/v1/P17-2008.

[29] Y. Zhu *et al.*, "Uncertainty-Guided Chain-of-Thought for Code Generation with LLMs," *ArXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2503.15341

[30] K. He *et al.*, "Towards Better Code Generation: Adaptive Decoding with Uncertainty Guidance," *ArXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2506.08980

[31] X. Wang *et al.*, "Compilable Neural Code Generation with Compiler Feedback," *ArXiv*, 2022, [Online]. Available: https://arxiv.org/abs/2203.05132

[32] G. Dolcetti *et al.*, "Helping LLMs Improve Code Generation Using Feedback from Testing and Static Analysis," *ArXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2412.14841

[33] K. Liu *et al.*, "LLM-Powered Test Case Generation for Detecting Bugs in Plausible Programs," *ArXiv*, 2025, [Online]. Available: https://arxiv.org/abs/2404.10304

[34] D. Guo *et al.*, "DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence," 2024, [Online]. Available: https://arxiv.org/abs/2401.14196

[35] B. Hui *et al.*, "Qwen2.5-Coder Technical Report," 2024, [Online]. Available: https://arxiv.org/abs/2409.12186

[36] C. Guo *et al.*, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 70. 2017, pp. 1321–1330.